23CSE455 – Design Pattern Case Study

Design Pattern Analysis and Evolution in a Ruby on Rails Hotel Booking Application

Final Academic Report

Team	A7
Programme / Batch	B.Tech Artificial Intelligence & Data Science (AI & DS)
Application	`rails-room-booking`
Source language	Ruby / Ruby on Rails
Part 3 languages	Java, Python, JavaScript, C++
Baseline commit	`5e15551ee646871b4bed32c982fa17da319d3ee3`
Submission date	6 September 2027

Team Members

Roll Number	Name	Batch
AM.AI.U4AID23016	Kavya Penikalapati	AI & DS
AM.AI.U4AID23037	E. Abhijith Krishna	AI & DS
AM.AI.U4AID23040	Guduru Akshaya Giri	AI & DS
AM.AI.U4AID23046	Kurt Sony Rebello	AI & DS

Prepared as an evidence-based software engineering case study.

Contents

1 Executive Summary **1**

2 Introduction **1**
2.1 Design patterns and evidence-based identification . . . 1
2.2 Case-study purpose . . . 2
2.3 Three-part assignment structure . . . 2

3 Application and Architecture **2**
3.1 Application background . . . 2
3.2 Baseline source and reproducibility . . . 2
3.3 Architecture overview . . . 3
3.4 Major framework and integration mechanisms . . . 4
3.5 Architecture collaborations . . . 4
3.5.1 User and Booking . . . 4
3.5.2 User and Room . . . 4
3.5.3 Room and Booking . . . 4
3.5.4 Booking and Setting . . . 4
3.5.5 Booking and Payify . . . 4
3.5.6 Booking and Integromat . . . 4
3.5.7 Controllers and CrudConcern . . . 4
3.5.8 Controllers and Pundit . . . 5
3.6 Important baseline flows . . . 5
3.6.1 Booking creation . . . 5
3.6.2 Availability . . . 5
3.6.3 Cancellation . . . 5
3.6.4 Payment . . . 5
3.6.5 Booking notification . . . 5

4 Part 1 – Pattern Discovery **5**
4.1 Discovery method . . . 5
4.2 Baseline result . . . 6
4.3 Template Method – detailed analysis . . . 6
4.3.1 Participant roles . . . 6
4.3.2 Why this is Template Method . . . 7
4.4 Concrete controller specialization . . . 7
4.5 Why this is more than ordinary reuse . . . 7
4.6 Rejected and uncertain candidates . . . 8

5 Part 2 – Evolution and Refactoring **9**
5.1 Requirement evolution . . . 9
5.2 Change 1 – Additional services . . . 9
5.3 Change 2 – Room upgrade . . . 10
5.4 Change 3 – Cancellation Strategy . . . 10
5.5 Why cancellation is genuinely Strategy . . . 11
5.6 Before and after architecture . . . 12
5.7 Pattern evolution . . . 13
5.8 Refactoring analysis . . . 13
5.9 Part 2 verification status . . . 13

6 Part 3 – Cross-language Comparison **14**
6.1 Purpose and scope . . . 14
6.2 Shared behavioral specifications . . . 14
6.3 Template Method mapping . . . 14
6.4 Strategy mapping . . . 14
6.5 Execution results . . . 15
6.6 Static comparison . . . 15
6.7 Performance measurements . . . 15
6.8 Language-specific observations . . . 15

7 LLM Records and Human Verification **16**
7.1 Role of LLM assistance . . . 16
7.2 Retained records . . . 16
7.3 Human verification . . . 16

8 Limitations **17**

9 Findings and Contributions **17**
9.1 Finding 1 – A single baseline pattern is defensible . . . 17
9.2 Finding 2 – False positives matter . . . 17
9.3 Finding 3 – Requirements create different design pressures . . . 17
9.4 Finding 4 – Cancellation is a strong Strategy case . . . 17
9.5 Finding 5 – Cross-language mechanisms differ . . . 18
9.6 Finding 6 – Evidence reduces pattern inflation . . . 18
9.7 Contributions . . . 18

10 Conclusion **18**

11 References **18**

A Detailed Source Evidence **19**
A.1 CrudConcern evidence . . . 19
A.2 Booking evidence . . . 19

B Part 2 Change Acceptance **20**
B.1 Change 1 acceptance . . . 20
B.2 Change 2 acceptance . . . 20
B.3 Change 3 acceptance . . . 20

C Cross-language Test Specification **20**

D Submission and Evidence Audit **20**

List of Figures

1 Baseline Template Method collaboration implemented by Ruby module inclusion. Dashed dependencies denote `include CrudConcern`, not class inheritance; controller hooks specialize the shared CRUD skeleton. . . . 3
2 Baseline application architecture reconstructed from the inspected Rails source. External integrations are shown as boundaries; their internal implementations are not assumed. . . . 3
3 Additional-service design for breakfast and airport transfer. . . . 9
4 Cancellation Strategy collaboration. . . . 11
5 Cancellation sequence showing context-to-policy delegation. . . . 12
6 Evolved architecture with additional services, room upgrade and cancellation policy. . . . 12

List of Tables

1 Baseline framework mechanisms and their role . . . 4
2 Baseline pattern inventory . . . 6
3 Template Method source evidence . . . 7
4 Rejected or uncertain baseline candidates . . . 8
5 Evolution requirements and design response . . . 9
6 Pattern evolution across the three changes . . . 13
7 Final Rails verification evidence . . . 13
8 Strategy behavioral specification . . . 14
9 Template Method mapping across languages . . . 14
10 Strategy mapping across languages . . . 14
11 Part 3 execution verification . . . 15
12 Cross-language static metrics . . . 15
13 Median process-level timing . . . 15
14 LLM evidence and verification status . . . 16
15 Final evidence audit . . . 20

1 Executive Summary

This case study investigates the architecture and design-pattern structure of `rails-room-booking`, an open-source Ruby on Rails hotel/room booking application. The study uses an immutable source baseline rather than treating the current repository state as an informal reference: the original application is identified by repository https://github.com/andrewdsilva/rails-room-booking and commit `5e15551ee646871b4bed32c982fa17da319d3ee3`. The investigation covers the application's domain model, controllers, Rails concerns, authentication and authorization mechanisms, payment integration, notification integration, and booking lifecycle.

The central methodological decision was to distinguish framework mechanisms and ordinary object-oriented reuse from application-level Gang of Four (GoF) design patterns. A pattern was confirmed only when its participant roles, interactions, source locations, and behavioral purpose could be established from the source. This prevents a common failure mode in design-pattern assignments in which every callback is called Observer, every policy class is called Strategy, or every wrapper is called Decorator or Adapter.

Under that evidence standard, the baseline contains **one clearly confirmed application-level GoF collaboration: Template Method**. The collaboration is the shared CRUD workflow implemented by `CrudConcern` and specialized by `BookingsController`, `RoomsController`, and `SettingsController`. `CrudConcern` provides the common algorithm through actions such as `index`, `show`, `new`, `create`, `edit`, `update`, and `destroy`; concrete controllers provide hook methods such as `base_class`, `object_class`, `strong_params`, `model_params`, `use_slug?`, `set_object`, and `redirect_to_show`. The use of explicit `NotImplementedError` extension points further demonstrates that the common workflow is intentionally parameterized by resource-specific operations.

The second part evolves the design for four realistic requirements: a breakfast package, an airport transfer, room upgrades, and configurable cancellation policies. The first two are modeled as optional additional services rather than permanent room attributes. Room upgrade is isolated as a reservation-domain operation. Cancellation is the strongest case for a new GoF pattern: a `CancellationService` delegates variable cancellation and refund behavior to a `CancellationPolicy`, with concrete policies for flexible, partial-refund, and no-refund rules. This establishes a genuine **Strategy** collaboration because the context delegates the same conceptual operation to interchangeable algorithms. The complete evolved Rails application passed 36 RSpec examples with zero failures, including 30 feature examples and an eager-loading check, after rebuilding the dedicated PostgreSQL test database from all 12 migrations.

The third part reproduces the two confirmed pattern ideas independently in Java, Python, JavaScript, and C++. The implementations do not attempt to port the entire Rails application. Instead, they preserve the pattern intent and representative behavior. Eight snippets were executed: four Template Method implementations and four Strategy implementations. Each passed its specified behavioral check. Process-level timing used one warmup followed by five measured runs, with the median reported. The measured medians show substantial runtime differences between languages, but these observations are environment-dependent and are not interpreted as universal language-speed claims.

LLM assistance was used as an analytical aid for source exploration, candidate pattern discovery, evolution reasoning, cross-language implementation, and final audit. The human-verification rule was retained throughout: an LLM hypothesis was not treated as evidence until checked against source code or executable artifacts. Historical Prompt 02 and Prompt 03 wording could not be recovered verbatim from the retained records, so this report explicitly records that limitation rather than fabricating a transcript.

The result is intentionally conservative. The project demonstrates that a smaller pattern inventory can be more academically defensible than a larger inventory produced by pattern-by-name classification. It also demonstrates that new requirements do not necessarily justify forcing every new class into a GoF pattern: additional services and room upgrade are primarily domain-modeling/refactoring decisions, while cancellation-policy variation has the structural and behavioral characteristics required for Strategy.

2 Introduction

2.1 Design patterns and evidence-based identification

Software design patterns are reusable descriptions of design structures that solve recurring problems in a context. Their value is not in assigning familiar names to arbitrary classes; the value lies in recognizing a recurring collaboration and understanding the trade-offs that motivate it. In an academic case study, this distinction is particularly important because a framework such as Ruby on Rails already provides conventions, lifecycle callbacks, inheritance, concerns, routing, persistence and dependency mechanisms. These mechanisms can resemble design-pattern vocabulary without necessarily constituting an application-level GoF pattern.

The present study therefore treats pattern identification as an evidence problem. A candidate must have recognizable participant roles, interactions between those participants, a behavioral or structural purpose corresponding to the pattern intent, and source evidence that demonstrates the collaboration. A class name such as `Policy` is not enough to prove Strategy; a callback is not automatically Observer; and a concern is not automatically Decorator.

2.2 Case-study purpose

The purpose of the case study is to investigate an existing hotel booking application, identify the patterns that can actually be defended from source evidence, evolve the design under new requirements, and compare representative implementations across four programming languages.

The study objectives are:

1. Analyze the architecture of an existing hotel booking application.
2. Identify design patterns actually present in the source.
3. Distinguish genuine patterns from superficial pattern candidates.
4. Evolve the application in response to new requirements.
5. Determine whether new design patterns become justified after evolution.
6. Implement representative confirmed patterns in Java, Python, JavaScript and C++.
7. Compare the implementations using structural, behavioral and process-level performance evidence.

These are study objectives, not claims that the expected outcomes were known before the investigation.

2.3 Three-part assignment structure

The investigation is divided into three parts. Part 1 establishes a reproducible baseline and performs pattern discovery. Part 2 introduces new domain requirements and evaluates the resulting design pressure, including a Strategy-based cancellation-policy design. Part 3 isolates the confirmed pattern collaborations into small executable examples so that the same design intent can be compared across four languages.

3 Application and Architecture

3.1 Application background

The studied application is `rails-room-booking`, an existing Ruby on Rails hotel/room booking application. The baseline source is the public repository https://github.com/andrewdsilva/rails-room-booking at commit `5e15551ee646871b4bed32c982fa17da319d3ee3`. The source language is Ruby and the application uses Rails MVC and Active Record as its main application architecture.

The baseline domain entities established directly from the inspected source are:

- **User**: authenticated application user and owner of rooms/bookings/settings.
- **Room**: bookable accommodation with descriptive attributes, capacity, pricing and amenity flags.
- **Booking**: reservation connecting a user and room, with dates, guests, price totals and cancellation state.
- **Setting**: user-associated configuration, including VAT used by booking-price calculation.

The baseline source does **not** establish separate application-owned models for Hotel, Customer, Cancellation, or AdditionalService. This distinction matters because the case study must not retroactively attribute domain objects to the application merely because those concepts appear in the assignment statement.

3.2 Baseline source and reproducibility

The immutable baseline is:

Repository	https://github.com/andrewdsilva/rails-room-booking
Baseline commit	`5e15551ee646871b4bed32c982fa17da319d3ee3`
Application language	Ruby
Framework	Ruby on Rails
Ruby version in baseline Gemfile	2.7.5
Rails dependency in baseline Gemfile	`~7.0.4`
Database at baseline commit	PostgreSQL

The database conclusion is supported by the baseline commit itself: that commit changes the application from SQLite to PostgreSQL, adds the `pg` dependency, changes `code/config/database.yml`, and adds a PostgreSQL service to Docker Compose. The source therefore supports describing PostgreSQL as the baseline database configuration.

The complete baseline source is retained under `code/`; the complete evolved Rails application is retained under `Part2/code/`. The inspected application areas include the main domain models, controllers and controller concerns, routing, schema, selected model specifications and the relevant dependency configuration. External-library internals were not treated as observable application source.

3.3 Architecture overview

The baseline follows the conventional Rails flow:

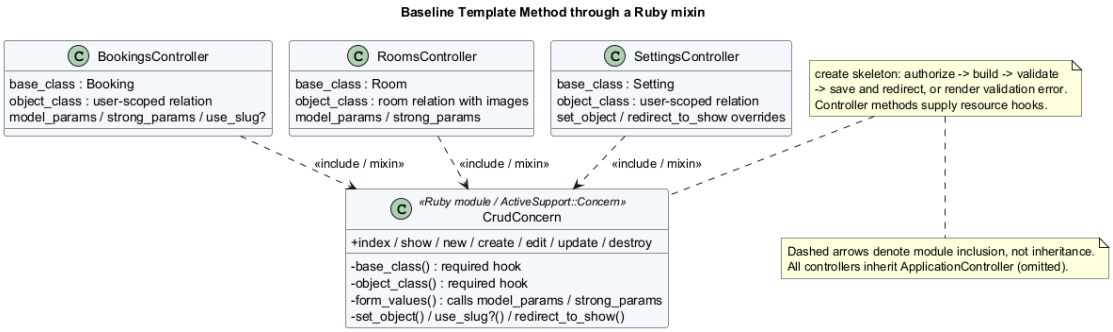


Figure 1: Baseline Template Method collaboration implemented by Ruby module inclusion. Dashed dependencies denote `include CrudConcern`, not class inheritance; controller hooks specialize the shared CRUD skeleton.

At a high level, a browser request reaches Rails routes and a controller. Controllers use Active Record models and model concerns to access and manipulate persistent state. PostgreSQL stores the application data. Pundit is used for authorization, Devise for authentication, Payify/Stripe integration for payments, Active Storage for attachments, and Integromat/Make integration for booking webhooks. Stimulus and Turbo participate in the front-end interaction model.

The architecture is therefore better described as a Rails MVC/Active Record application with reusable concerns and integrations than as a collection of GoF patterns. This framing is important because Rails itself supplies substantial structure.

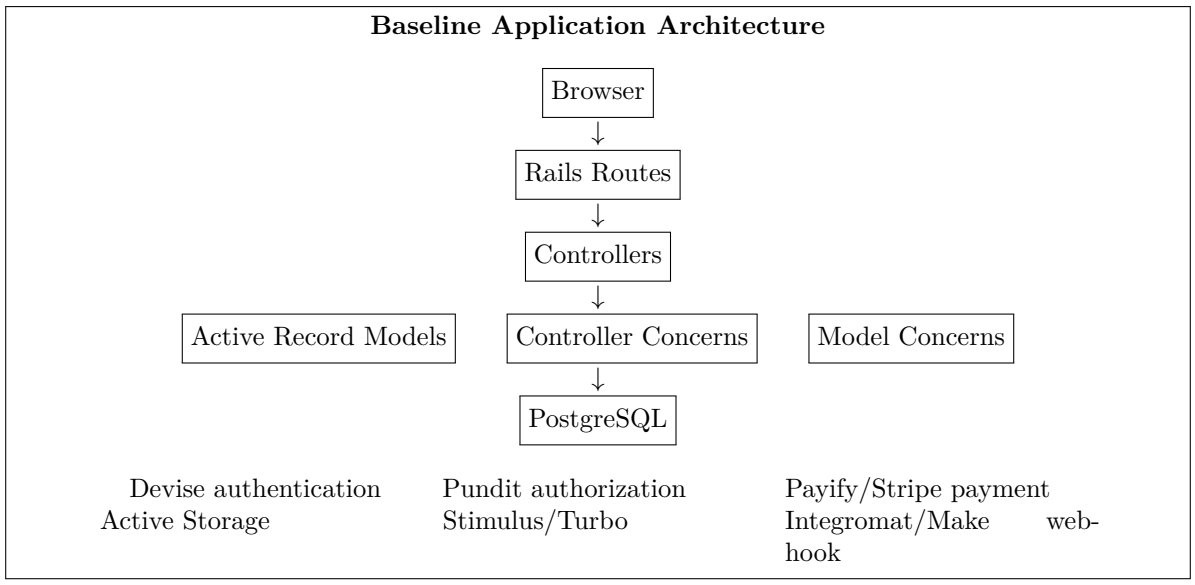


Figure 2: Baseline application architecture reconstructed from the inspected Rails source. External integrations are shown as boundaries; their internal implementations are not assumed.

3.4 Major framework and integration mechanisms

Table 1: Baseline framework mechanisms and their role

Mechanism	Evidence-backed role
Rails MVC	Routes requests to controllers and renders/redirects responses.
Active Record	Maps application models to persistent relational data and provides associations/querying.
Devise	Authentication/user-account mechanism referenced by the application.
Pundit	Authorization mechanism used by controllers through `authorize`.
Payify / Stripe	Payment integration visible through the Booking payment concern and payment controller/engine.
Active Storage	Room image attachments.
Integromat / Make	Booking webhook integration visible in `Booking#notify_booking`.
Stimulus / Turbo	Front-end interaction mechanisms included by the Rails application.
Controller concerns	Reusable controller behavior, including the shared CRUD workflow.
Model concerns	Reusable model behavior, including pricing/ownership/settings-related logic.

The table deliberately labels these as framework or integration mechanisms. Their presence does not automatically imply a corresponding GoF pattern.

3.5 Architecture collaborations

3.5.1 User and Booking

The source establishes `Booking belongs_to :user` and `User has_many :bookings`. The booking controller also scopes booking queries by `current_user.id`. This provides both persistence-level association and controller-level ownership scoping.

3.5.2 User and Room

The source establishes `Room belongs_to :user` and `User has_many :rooms`. This relationship supports room ownership and the room-management scope.

3.5.3 Room and Booking

The source establishes `Room has_many :bookings` and `Booking belongs_to :room`. Availability is checked in `Booking#dates_are_available`, where conflicting bookings are queried using the candidate booking's dates. `Room#booked_dates` separately exposes future booked date ranges in a display-oriented representation.

3.5.4 Booking and Setting

Pricing behavior is extracted into `HasPriceConcern`. Its `vat` method accesses `room.user.setting.vat`, making the setting part of the booking-price calculation.

3.5.5 Booking and Payify

`Booking` includes `Payify::HasPaymentConcern`. The source also defines `payment_pending?`, `paid?`, `amount_to_pay` and the `stripe_init_intent` controller hook. The internal implementation of Payify/Stripe is not treated as part of the observable application pattern evidence.

3.5.6 Booking and Integromat

The Booking model contains `after_create :notify_booking`. The `notify_booking` method checks for the configured booking webhook and invokes `Integromat::Webhook.new(:booking).trigger(...)`. This establishes an observable integration, but it does not by itself prove an Observer pattern.

3.5.7 Controllers and CrudConcern

The controllers include `CrudConcern`, a Ruby module using `ActiveSupport::Concern`. The module is mixed into each controller; it is not a superclass, and no `CrudTemplate` interface exists in the source. Its public CRUD actions preserve the algorithmic skeleton while private helper methods and controller hook overrides supply resource-specific behavior. All three controllers inherit `ApplicationController` independently of module inclusion. This collaboration is the strongest baseline GoF pattern evidence.

3.5.8 Controllers and Pundit

The CRUD workflow and the booking cancellation action call `authorize`. This demonstrates an authorization collaboration, but the available source does not establish the participant structure required to call the Pundit policy hierarchy a Strategy instance in the baseline.

3.6 Important baseline flows

3.6.1 Booking creation

The observable booking-creation path is:

```
Request -> Route -> BookingsController -> CrudConcern#create -> authorize -> Booking.new -> validation ->
                                        save -> callbacks -> redirect
```

The controller delegates the common create algorithm to the concern. Booking-specific behavior enters through `object_class`, `strong_params`, `model_params`, and the model's own validations/callbacks.

3.6.2 Availability

The availability validation is performed by `Booking#dates_are_available`. It queries the room's existing bookings and rejects a new booking when date intervals overlap.

Listing 1: Verified baseline excerpt: booking availability validation

```
1 def dates_are_available
2   conflicting_bookings = room.bookings.where.not(id: id).
3     where("(?␣<␣end_date)␣AND␣(?␣>␣start_date)", start_date, end_date)
4
5   errors.add(:base, I18n.t(
6     "activerecord.errors.models.booking.unavailable_dates"
7   )) if conflicting_bookings.exists?
8 end
```

3.6.3 Cancellation

The baseline cancellation flow is:

```
POST /bookings/:id/cancel -> BookingsController#cancel -> authorize -> Booking#cancel!  -> canceled_at ->
                                                        redirect
```

The baseline implementation simply records cancellation time through `update_attribute :canceled_at, Time.now`. There is no application-owned cancellation-policy hierarchy in the baseline source.

3.6.4 Payment

Payment behavior is partially delegated to the Payify concern. The Booking model includes the concern, and the controller invokes a Stripe initialization operation when the payment is pending. Because Payify/Stripe internals are external, the report does not claim an application-level Adapter, Facade or Strategy pattern inside those dependencies.

3.6.5 Booking notification

```
Booking creation -> after_create -> notify_booking -> Integromat webhook
```

This is an observable callback-to-integration path. It is intentionally classified as a framework/application callback mechanism rather than Observer because the required observer/subject collaboration is not established by the inspected source.

4 Part 1 – Pattern Discovery

4.1 Discovery method

The pattern-discovery method follows a source-first procedure:

1. Establish an immutable source baseline.

2. Identify candidate collaborations from controllers, models, concerns and service boundaries.
3. Trace calls and overrides between candidate participants.
4. Map the observed roles to the recognized pattern intent.
5. Record source locations and behavioral purpose.
6. Reject candidates when required participant/delegation evidence is absent.
7. Verify the final inventory independently against the source.

The central methodological rule is:

> A pattern is not confirmed from class names, annotations, inheritance, Rails conventions, or an LLM assertion alone. Confirmation requires participant roles, actual source locations, actual interactions, delegation/calls, behavioral purpose, and correspondence with the recognized pattern.

This produces a smaller but more defensible inventory.

4.2 Baseline result

The baseline investigation confirmed exactly **one GoF pattern type and one pattern instance**:

Table 2: Baseline pattern inventory

Field	Verified result
Pattern	Template Method
Category	Behavioral
Instance	Shared CRUD Controller Workflow
Template holder	`CrudConcern`
Concrete participants	`BookingsController`, `RoomsController`, `SettingsController`
Count of confirmed pattern types	1
Count of confirmed pattern instances	1

4.3 Template Method – detailed analysis

4.3.1 Participant roles

The template holder is `CrudConcern`. The concrete participants are `BookingsController`, `RoomsController`, and `SettingsController`.

The concern provides the common CRUD algorithm:

- `index`
- `show`
- `new`
- `create`
- `edit`
- `update`
- `destroy`

The extension points include:

- `base_class`
- `object_class`
- `strong_params`
- `model_params`
- `use_slug?`

- `set_object`
- `redirect_to_show`

The baseline source locations are:

Table 3: Template Method source evidence

Participant	Verified source range
Template holder: `CrudConcern`	`code/app/controllers/concerns/crud_concern.rb:1-149`
Concrete: `BookingsController`	`code/app/controllers/bookings_controller.rb:1-47`
Concrete: `RoomsController`	`code/app/controllers/rooms_controller.rb:1-41`
Concrete: `SettingsController`	`code/app/controllers/settings_controller.rb:1-39`

4.3.2 Why this is Template Method

The collaboration has the essential shape of Template Method even though Ruby/Rails expresses it through a module concern rather than a Java-style abstract class.

First, the reusable algorithm is centralized. For example, `create` performs authorization, constructs an object, validates it, saves it, and redirects on success; otherwise it renders the form with an error status. The algorithm is not copied separately into each resource controller.

Second, the algorithm invokes resource-specific hooks. `object_class`, `strong_params` and `model_params` determine which resource and parameters participate in the shared lifecycle.

Third, the concrete controllers implement those hooks differently. Their `base_class` methods return `Booking`, `Room`, and `Setting`, respectively; their `object_class` methods return the corresponding scoped Active Record relations. `SettingsController` additionally overrides `set_object` and `redirect_to_show` because its resource semantics differ.

Fourth, the concern contains explicit required extension points. `base_class` and `object_class` raise `NotImplementedError` in the shared module. This is stronger evidence than mere code reuse: it establishes a reusable algorithm with designated operations that concrete participants must supply.

Listing 2: Verified baseline Template Method hook structure

```
def base_class
  raise NotImplementedError
end

def object_class
  raise NotImplementedError
end

def use_slug?
  true
end
```

The pattern therefore represents a deliberate collaboration: a fixed workflow is preserved while resource-specific details vary.

4.4 Concrete controller specialization

`BookingsController` specializes the template with Booking-specific querying, permitted parameters and non-slug lookup. It also adds a booking-specific cancellation action and Stripe payment initialization.

`RoomsController` specializes the template with Room-specific querying, image attachment handling and room parameters. It also adds booking-form initialization for a room.

`SettingsController` specializes the template with Setting-specific lookup and redirects. Its `set_object` implementation obtains the current user's setting and its `redirect_to_show` redirects to the settings path rather than a polymorphic resource show page.

This variation is exactly why the shared algorithm is useful: the common CRUD sequence is stable while resource-specific behavior is localized.

4.5 Why this is more than ordinary reuse

A generic helper function could reduce duplication without being Template Method. The distinguishing evidence here is that the shared concern owns the workflow and explicitly defines a family of operations that concrete

controllers supply or override. The concrete classes are not simply calling a utility; they participate in a common algorithmic skeleton.

This distinction is important in the academic classification. The report therefore identifies Template Method rather than simply saying that `CrudConcern` is "reusable code."

4.6 Rejected and uncertain candidates

Table 4: Rejected or uncertain baseline candidates

Candidate	Initial suggestion	Reason not confirmed
Strategy via Pundit	Policies and `authorize` suggest policy-based behavior.	The inspected application does not establish a Strategy context delegating a common operation to interchangeable policy algorithms.
Strategy via CrudConcern hooks	Hook methods vary by controller.	The variation belongs to the Template Method mechanism; the shared CRUD algorithm is not selecting interchangeable algorithms at runtime.
Policy inheritance	Pundit policies form an authorization structure.	Inheritance alone is insufficient; the required interchangeable algorithm collaboration was not established.
Observer	`after_create` and `after_save` callbacks trigger side effects.	Rails callbacks are lifecycle hooks; the source does not establish a subject maintaining and notifying observer objects.
Adapter	Payify/Stripe integration looks like an external interface boundary.	Payify internals are external and not observable as an application-owned adapter collaboration.
Facade	Payment functionality is accessed through a higher-level concern.	No application-owned facade abstraction was established from the inspected source.
Decorator	Rails concerns wrap/add model behavior.	The concern mechanism is not demonstrated as runtime wrapping of component objects.
Composite	Active Record associations form object relationships.	Associations do not establish the recursive uniform part-whole structure required by Composite.
Factory Method / Abstract Factory	Rails constructors and persistence helpers create objects.	No dedicated application-owned factory collaboration with varying creation products was established.
Command	Controller actions represent operations.	A controller action is not automatically a Command object with encapsulated request/operation state.
State	Booking has states such as paid/pending/canceled.	No state-object hierarchy or state-dependent polymorphic delegation is present in the baseline.
Memento	Booking persists state.	Persistence alone is not an object-state snapshot/restore collaboration.
Chain of Responsibility	Request processing passes through Rails layers.	Framework request flow does not establish a chain of interchangeable handlers in the application code.
Prototype	Active Record objects can be duplicated.	No prototype-based object creation mechanism is established.
Singleton	Rails configuration objects are globally accessible.	Framework singleton-like services are not evidence of an application-owned Singleton implementation.

The rejected-candidate analysis is not a claim that these patterns are impossible in Rails. It is a claim about what can be defended from this particular baseline source.

5 Part 2 – Evolution and Refactoring

5.1 Requirement evolution

The evolved requirements are:

1. Breakfast package.
2. Airport transfer.
3. Room upgrade.
4. Configurable cancellation policy.

These requirements create three different categories of design pressure:

- **Optional services**: breakfast and airport transfer.
- **Core reservation modification**: room upgrade.
- **Business-rule variation**: cancellation policy.

This distinction is important because not every new requirement needs a new GoF pattern.

Table 5: Evolution requirements and design response

Change	Requirement	Design response
1	Breakfast and airport transfer	Additional-service abstraction and booking-service association
2	Room upgrade	Dedicated room-upgrade domain operation with availability and price-difference logic
3	Cancellation policy	Cancellation context plus Strategy-based policy hierarchy

5.2 Change 1 – Additional services

Breakfast and airport transfer are optional booking-level services. Representing each as a permanent attribute of `Room` or `Booking` would increase conditional logic and make every new service require schema and business-rule changes to the core booking object.

The evolved design introduces an `AdditionalService` base class, with concrete `BreakfastPackage` and `AirportTransfer` services. The base class is instantiable rather than a Ruby abstract class. `BookingService` records belong to a booking and store a captured `service_type`, `quantity` and `unit_price`; they do not have an Active Record association to an `AdditionalService` object. Their `subtotal` method multiplies captured price by quantity, and `AdditionalServicePricing.total` sums those record subtotals.

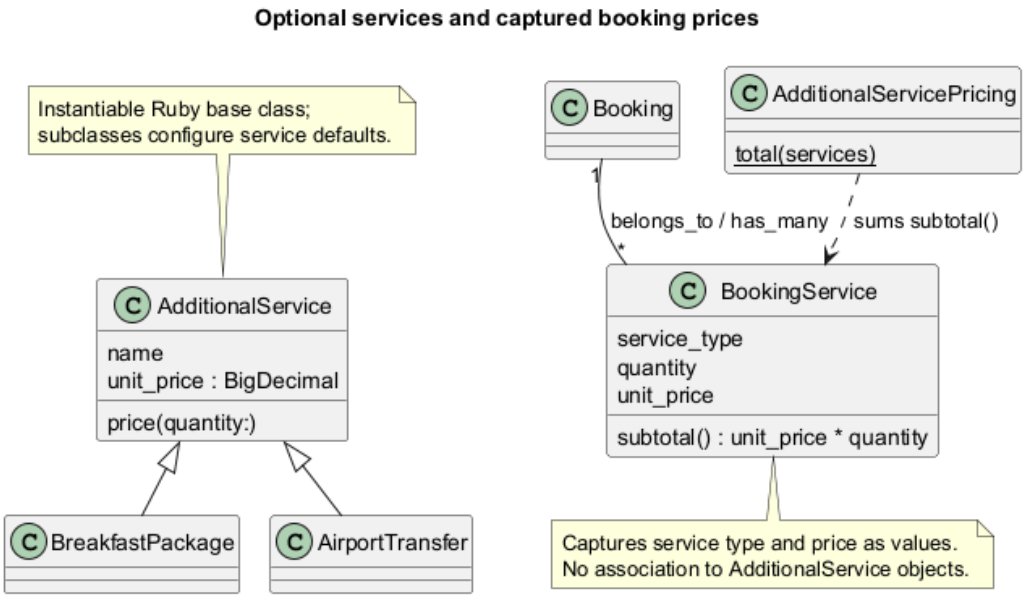


Figure 3: Additional-service design for breakfast and airport transfer.

The key design decision is domain modeling, not pattern labeling. The abstraction allows optional services to grow without turning the core booking record into a list of unrelated boolean or numeric attributes.

The implementation also separates pricing calculation from the controller workflow. The booking total can be expressed as the base accommodation amount plus the subtotal of selected services. This makes service pricing a domain concern that can be tested independently.

The design is **not** classified as Decorator, Composite, Strategy or Factory. Although the service abstraction could be combined with other patterns in a larger system, the implemented structure does not require those pattern claims.

5.3 Change 2 – Room upgrade

A room upgrade differs from an additional service because it changes the core reservation target. The operation must:

1. identify the requested target room;
2. verify that the target room is available for the booking interval;
3. calculate the nightly price difference;
4. change the booking's room;
5. recalculate the booking totals;
6. persist the updated booking; and
7. retain a link to the first original room through `upgraded_from_room`.

The implementation isolates this workflow in `RoomUpgrade`. The service receives a booking and target room, validates the requested change, computes the difference, applies the new room and persists it. Saving the booking runs the pricing callbacks that recalculate totals, including selected service charges. The retained original-room association survives repeated upgrades; it is a reference to the first room, not a complete event-history ledger.

This avoids adding a large conditional branch to `Booking#cancel!` or to controller actions. It also keeps the semantic distinction between "an optional service attached to a booking" and "a replacement of the room being booked."

The operation is not classified as Command, State or Strategy. The presence of an operation object does not automatically create Command; the implementation is primarily a domain responsibility extraction.

5.4 Change 3 – Cancellation Strategy

Cancellation is different because the business rule can vary while the overall cancellation workflow remains stable. A hotel may offer flexible cancellation, partial refund, or no refund. The core application should not need a growing conditional tree such as:

Listing 3: Illustrative anti-pattern avoided by the evolution

```
if policy == "flexible"
  ...
elsif policy == "partial"
  ...
elsif policy == "none"
  ...
end
```

Instead, the evolved design uses:

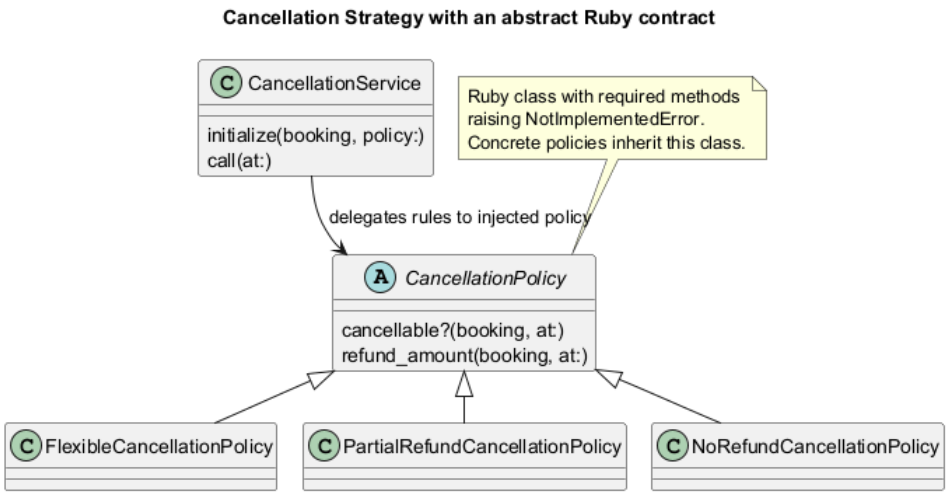


Figure 4: Cancellation Strategy collaboration.

The context is `CancellationService`. The Strategy interface is `CancellationPolicy`. The concrete strategies are:

- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

The context delegates two conceptually related business decisions: whether cancellation is permitted and how the refund is calculated. The policy object therefore encapsulates the variable algorithm. The real `BookingsController#cancel` action authenticates the user, retrieves the scoped booking, authorizes it, and obtains a policy from server configuration. The client cannot select a more generous policy through request parameters. `CancellationService#call` locks the booking, rejects repeated cancellation, delegates the rules, and atomically persists `canceled_at` with `cancellation_refund_amount`. This records a calculated refund; it does not issue a Stripe refund request or alter the paid payment record.

Listing 4: Evolved Strategy context and delegation from the final source

```
class CancellationService
  def initialize(booking, policy:)
    @booking = booking
    @policy = policy
  end

  def call(at: Time.current)
    @booking.with_lock do
      raise ArgumentError, "Booking␣is␣already␣canceled" if @booking.canceled?
      raise ArgumentError, "Cancellation␣is␣not␣permitted" unless @policy.cancellable?(@booking, at:
    at)

      refund = @policy.refund_amount(@booking, at: at)
      @booking.update!(canceled_at: at, cancellation_refund_amount: refund)
      { booking: @booking, refund_amount: refund }
    end
  end
end
```

5.5 Why cancellation is genuinely Strategy

The classification follows the GoF intent rather than the class names:

1. There is a family of related algorithms: cancellation/refund rules.
2. The algorithms perform the same conceptual operation for a common context.
3. The context can receive different policy implementations.
4. The context delegates the variable behavior instead of containing all policy branches.

5. A new policy can be added without changing the central cancellation orchestration.

The Strategy collaboration is therefore substantially stronger than the rejected baseline Pundit-as-Strategy candidate.

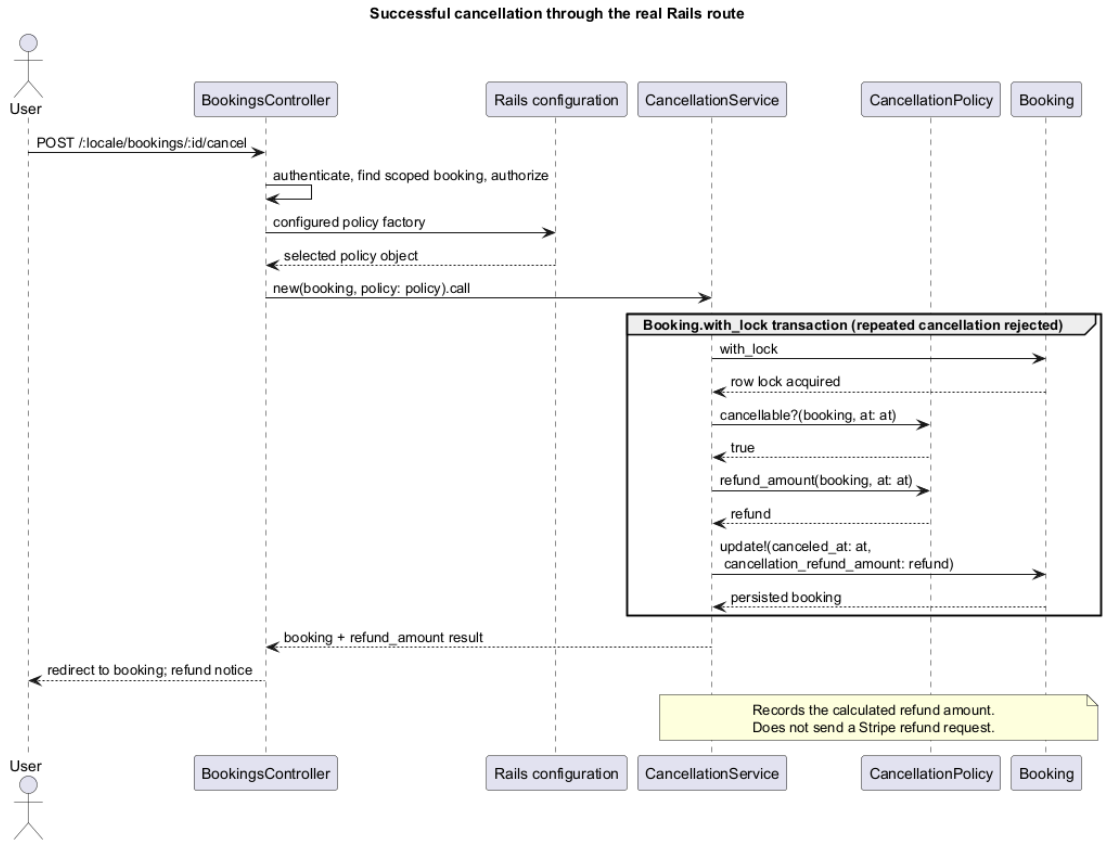


Figure 5: Cancellation sequence showing context-to-policy delegation.

5.6 Before and after architecture

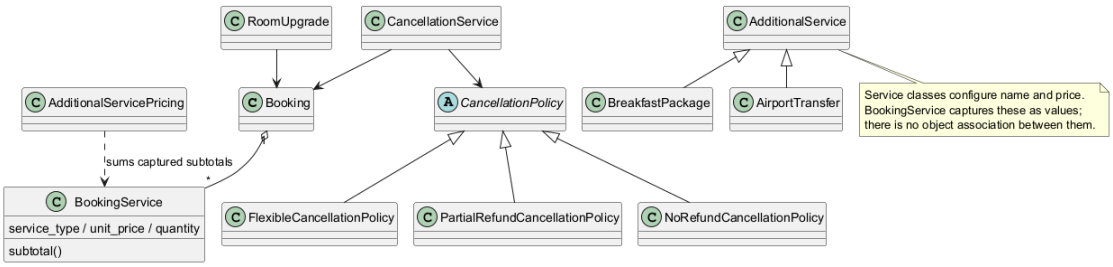


Figure 6: Evolved architecture with additional services, room upgrade and cancellation policy.

The baseline architecture centered most reservation behavior around `Booking` and reusable Rails concerns. The evolved architecture introduces explicit application-service/domain boundaries for the new requirements. This is a responsibility-separation decision rather than a claim that every new class is a design pattern.

5.7 Pattern evolution

Table 6: Pattern evolution across the three changes

Version	Pattern	Instance	Status
Original	Template Method	Shared CRUD Controller Workflow	Baseline confirmed
After Change 1	Template Method	Shared CRUD Controller Workflow	Preserved
After Change 2	Template Method	Shared CRUD Controller Workflow	Preserved
After Change 3	Template Method	Shared CRUD Controller Workflow	Preserved
After Change 3	Strategy	Cancellation Refund Policy	Newly introduced

The package's evolution CSV records Template Method as preserved and Strategy as newly introduced. `Part2/code/` contains the full application with the new models, migrations, application services, server-configured policy selection and real cancellation-route integration. The existing CRUD concern and its resource-specific hooks remain present.

5.8 Refactoring analysis

The primary design pressure is responsibility growth. The original Booking model already combines associations, availability validation, cancellation state, pricing behavior via a concern, payment integration and notification behavior. New requirements would increase this concentration if implemented directly in the model and controllers.

The evolution responds by extracting responsibilities:

- additional-service selection and pricing become explicit domain concepts;
- room-upgrade validation and price adjustment become a dedicated operation;
- cancellation rules become interchangeable policies;
- the existing controller Template Method remains responsible for the common CRUD lifecycle.

The resulting design is not claimed to be universally superior. It introduces more classes and indirection. That trade-off is justified only when the business rules are expected to vary or grow.

5.9 Part 2 verification status

The final application was tested with Ruby 2.7.5, Rails 7.0.6, Bundler 2.4.17 and PostgreSQL 15.10. A dedicated test database was recreated from scratch, all 12 migrations completed, and the complete RSpec suite passed **36 examples, zero failures**. This total comprises five baseline model examples, 30 feature examples and one eager-loading example. The three feature receipts report Change 1: 7/0, Change 2: 8/0, and Change 3: 15/0 (examples/failures). Change 3 includes nine cancellation service examples and six requests through the real Rails POST route.

Table 7: Final Rails verification evidence

Verification	Retained evidence
Complete RSpec suite: 36/0	`test_result/part2_rspec.txt`
Clean migration rebuild	`test_result/test_db_migrations.txt`
Twelve migrations up	`test_result/test_db_migration_status.txt`
Independent Rails runner boot	`test_result/rails_boot.txt`: Rails 7.0.6, test environment, 11 tables, exit zero
Real Puma HTTP startup	`test_result/rails_http_boot.txt`: GET / returns HTTP 302 with location /fr
Feature tests by change	`Part2/changes/Change1/test_result.txt`
	`Part2/changes/Change2/test_result.txt`
	`Part2/changes/Change3/test_result.txt`
Regression before fixes: 30/27	`test_result/part2_regression_before.txt`
Boot regression before fix: 1/1	`test_result/boot_regression_before.txt`

The suite exercises persisted room upgrades, original-room retention, availability boundaries, service totals, cancellation state/refund recording, configured policy selection, authentication, ownership and repeated cancellation. The eager-loading check required explicitly declaring `active_model_serializers`, a dependency used by the upstream Payify integration. The retained logs contain duplicate-constant warnings from `net/protocol`; the successful test and migration commands exit with status zero. These results establish the tested application behavior on the dedicated test database; production deployment and external payment/webhook delivery are outside that verification scope.

6 Part 3 – Cross-language Comparison

6.1 Purpose and scope

The cross-language exercise does not attempt to port the complete Rails application. It isolates the two confirmed design ideas and reproduces their representative behavior:

- Template Method: a shared resource-rendering/CRUD-style workflow with resource-specific hooks.
- Strategy: cancellation refund behavior selected through a policy object.

The purpose is to compare how different languages express abstraction, inheritance, interfaces, dynamic dispatch and ownership while preserving the same design intent.

6.2 Shared behavioral specifications

For Template Method, the common workflow creates a resource-specific representation using hooks for Booking, Room and Settings. The common algorithm remains in the template while the concrete resource implementations supply details.

For Strategy, a booking amount of 430 is passed to three policies. The expected outputs are:

Table 8: Strategy behavioral specification

Policy	Rule	Expected refund
Flexible	Full eligible refund	430
Partial refund	50% refund	215
No refund	No refund	0

All four language implementations use the same representative behavior.

6.3 Template Method mapping

Table 9: Template Method mapping across languages

Language	Main abstraction	Language-specific expression
Ruby	`CrudConcern`	ActiveSupport concern with shared workflow and hook methods.
Java	Abstract/template workflow class	Explicit abstract methods/interfaces and static typing.
Python	ABC-based workflow	Abstract base class and dynamic dispatch.
JavaScript	Base workflow class	Runtime polymorphism through class methods.
C++	Abstract base class	Pure virtual functions and virtual dispatch.

The language implementations are analogous, not source-identical. Ruby's concern mechanism is especially different from the other four because the original application integrates shared behavior through a module included into controllers.

6.4 Strategy mapping

Table 10: Strategy mapping across languages

Language	Strategy abstraction	Concrete policies
Java	Policy interface/abstract contract	Flexible, PartialRefund, NoRefund
Python	ABC-based policy contract	Flexible, PartialRefund, NoRefund
JavaScript	Base policy class	Flexible, PartialRefund, NoRefund
C++	Abstract policy class / shared ownership	Flexible, PartialRefund, NoRefund

The common structure is:

```
CancellationService -> CancellationPolicy -> Flexible / PartialRefund / NoRefund
```

The important cross-language invariant is delegation: the service does not encode each refund algorithm directly.

6.5 Execution results

Eight snippets were executed: Template Method and Strategy in each of four languages.

Table 11: Part 3 execution verification

Pattern	Language	Command family	Result
Template Method	Java	`javac Main.java && java Main`	Passed 1/1
Template Method	Python	`python3 Main.py`	Passed 1/1
Template Method	JavaScript	`node Main.js`	Passed 1/1
Template Method	C++	`g++ -std=c++17 ... && ./main`	Passed 1/1
Strategy	Java	`javac Main.java && java Main`	Passed 1/1
Strategy	Python	`python3 Main.py`	Passed 1/1
Strategy	JavaScript	`node Main.js`	Passed 1/1
Strategy	C++	`g++ -std=c++17 ... && ./main`	Passed 1/1

The test results are behavioral smoke checks for the isolated snippets, not proof that the complete Rails application behaves identically across languages.

6.6 Static comparison

The official cross-language dataset reports the following source LOC and maximum function cyclomatic complexity:

Table 12: Cross-language static metrics

Pattern	Language	Source LOC	Max function CC
Template Method	Java	9	1
Template Method	Python	17	1
Template Method	JavaScript	9	1
Template Method	C++	9	2
Strategy	Java	6	1
Strategy	Python	14	1
Strategy	JavaScript	6	1
Strategy	C++	9	2

LOC excludes blank lines and comments according to the project measurement convention. The low cyclomatic complexity reflects that the examples intentionally isolate pattern structure rather than application-scale business logic.

6.7 Performance measurements

Each snippet was measured using one warmup process followed by five measured process runs. The median process time is reported. Timing includes process startup and snippet execution.

Table 13: Median process-level timing

Language	Template Method (ms)	Strategy (ms)
Java	39.494	50.372
Python	672.008	595.258
JavaScript	21.401	21.762
C++	1.097	1.223

These measurements should not be interpreted as a universal language benchmark. The examples are tiny, and process startup can dominate their runtime. The useful conclusion is narrower: the same structural pattern can be implemented and executed in all four languages, while language/runtime characteristics produce different observed process-level timings.

6.8 Language-specific observations

Java. Java makes the pattern contracts explicit through interfaces and abstract classes. Static typing makes participant relationships visible in declarations, at the cost of more syntactic structure.

Python. Python can express the same abstraction using ABCs, but its dynamic dispatch makes some contracts less syntactically prominent. The implementation is concise in conceptual terms while retaining explicit abstract methods.

JavaScript. JavaScript's class syntax provides a familiar object-oriented surface, but dispatch remains dynamic. The pattern can be expressed without compile-time interface enforcement.

C++. C++ expresses the abstraction through pure virtual functions and runtime virtual dispatch. The implementation can also use ownership mechanisms such as `shared_ptr` for the selected Strategy.

7 LLM Records and Human Verification

7.1 Role of LLM assistance

LLMs were used as analytical assistants in several project stages:

1. baseline source analysis;
2. architecture extraction;
3. pattern discovery;
4. evolution analysis;
5. cross-language implementation;
6. final audit.

The methodological boundary was that an LLM output was treated as a hypothesis rather than as primary evidence.

7.2 Retained records

The package retains six analysis stages. Prompt 01 and Prompts 04–06 are preserved as prompt records. The exact historical wording for Prompts 02 and 03 could not be recovered as independent verbatim records. Rather than reconstructing those prompts and presenting the reconstruction as historical evidence, the package records the limitation.

Historical model/date metadata is also marked unknown where it could not be independently verified.

Table 14: LLM evidence and verification status

Stage	Retained evidence	Verification
Pattern discovery	Source-analysis result and evidence matrix	Source locations checked independently.
Evolution	Evolution analysis, change diffs and Rails receipts	Full Rails suite: 36 examples, zero failures.
Cross-language	Four-language implementations and result analysis	Eight snippets independently executed.
Final audit	Audit result and submission checklist	Package structure and evidence cross-checked.
Prompt 02/03 history	Verbatim historical records unavailable	Explicitly not reconstructed.

7.3 Human verification

Verification consisted of source inspection, participant mapping, candidate rejection, code execution where possible, and numerical cross-checking.

The most important human-verification outcomes were:

- Template Method was confirmed from concrete source interactions.
- Pundit was not inflated into Strategy.
- callbacks were not inflated into Observer.
- Payify was not inflated into Adapter/Facade.

- concerns were not inflated into Decorator.
- Part 2 Strategy was implemented as actual delegation rather than inferred from a class name.
- Part 3 timing values were collected rather than invented.

8 Limitations

A defensible case study must state the boundaries of its evidence.

1. **External payment internals.** Payify/Stripe internals are not fully observable from the application source, so their internal design patterns are not classified.
2. **External webhook internals.** Integromat/Make behavior outside the application's webhook invocation is not observable.
3. **Representative Part 3 scope.** The four-language examples are representative pattern implementations, not full Rails ports.
4. **Simplified business behavior.** Cross-language examples simplify application-scale concerns such as persistence, date availability, money representation and cancellation timing.
5. **Test environment scope.** Migrations, persisted feature behavior, model tests, actual-route requests and eager loading were verified on a dedicated PostgreSQL test database. Production deployment and live external-service operation were not exercised.
6. **Refund delivery scope.** Cancellation persists the calculated refund amount and timestamp. It does not issue an external Stripe refund or claim settlement of money.
7. **Timing scope.** Process-level timings include startup and are based on tiny programs. They are comparative observations, not language benchmarks.
8. **Historical LLM metadata.** Some historical model/date information is unavailable and therefore remains explicitly unknown.
9. **Historical Prompt 02/03 wording.** Exact verbatim records were not retained and were not reconstructed.
10. **Team contribution evidence.** Kurt Sony Rebello's primary contribution is documented in the project brief. The retained records do not independently document specific technical contributions for the other three students; the report therefore does not fabricate task assignments for them.

9 Findings and Contributions

9.1 Finding 1 – A single baseline pattern is defensible

The original application contains one clearly evidenced application-level GoF collaboration: Template Method through `CrudConcern` and its concrete controllers. This conclusion is narrower than a pattern-by-name inventory but stronger because the participant roles and calls can be traced directly.

9.2 Finding 2 – False positives matter

Several Rails mechanisms resemble classic patterns. However, similarity of vocabulary is insufficient. Pundit policies are authorization constructs; callbacks are lifecycle hooks; concerns are a Ruby/Rails reuse mechanism; and Payify is an external payment dependency. None was promoted to a GoF pattern without the required collaboration evidence.

9.3 Finding 3 – Requirements create different design pressures

Breakfast and airport transfer are best understood as optional service domain concepts. Room upgrade is a reservation modification workflow. Cancellation policy is a variable business-rule family. Treating these requirements identically would obscure their different design pressures.

9.4 Finding 4 – Cancellation is a strong Strategy case

The cancellation requirement creates interchangeable algorithms with a stable context. `CancellationService` delegates to `CancellationPolicy`, allowing flexible, partial-refund and no-refund implementations. This is the strongest new pattern introduced by the evolution.

9.5 Finding 5 – Cross-language mechanisms differ

The same design intent appears through Java interfaces/abstract classes, Python ABCs, JavaScript class-based dynamic dispatch and C++ pure virtual functions. The pattern concept is therefore more stable than its language-level expression.

9.6 Finding 6 – Evidence reduces pattern inflation

The project demonstrates a general methodological lesson: a defensible design-pattern inventory should be willing to contain only one baseline pattern when only one collaboration is actually established. Academic quality is improved by rejecting unsupported candidates rather than maximizing the pattern count.

9.7 Contributions

The project brief identifies Kurt Sony Rebello as the primary contributor for the overall case-study work, including source repository investigation, architecture analysis, pattern investigation, source-code verification, Part 1 identification, Part 2 evolution/design analysis and coordination, cancellation Strategy design, four-language implementation, testing/verification coordination, metrics, LLM-assisted analysis, report preparation, repository organization and final submission preparation.

For Kavya Penikalapati, E. Abhijith Krishna and Guduru Akshaya Giri, the retained project records available for this report do not contain independently verified, specific technical contribution assignments. Their names and team membership are therefore reported exactly, but no unsupported technical tasks are attributed to them.

10 Conclusion

This case study examined an existing Ruby on Rails hotel booking application using a source-first, evidence-based design-pattern methodology. The baseline was fixed to an immutable commit so that the pattern inventory could be reproduced against a defined source state. The architecture was analyzed as a Rails MVC/Active Record system with controllers, models, concerns and external integrations.

The baseline result was intentionally conservative: one application-level GoF collaboration, Template Method, was confirmed. `CrudConcern` provides a common CRUD algorithm and concrete resource controllers supply resource-specific hooks. The source locations, interactions and behavioral purpose support the classification. Several other plausible candidates were investigated but rejected because the required pattern collaboration was not established.

The evolution phase demonstrated that new requirements do not imply a uniform pattern solution. Breakfast and airport transfer were modeled as additional services. Room upgrade was isolated as a domain operation. Cancellation policy introduced genuine algorithmic variation and was therefore represented using Strategy. `CancellationService` acts as the context and delegates cancellation/refund behavior to interchangeable policy implementations.

The cross-language phase showed that the same design intent can be represented in Java, Python, JavaScript and C++. All eight representative snippets passed their behavioral checks. Process-level timing measurements were collected using one warmup and five measured runs, and the medians were reported. The measurements demonstrate environmental/runtime differences but do not establish universal language performance rankings.

The most important lesson is methodological. Design patterns should be identified from collaborations and intent, not from names or superficial structural resemblance. Likewise, evolution should introduce a pattern only when the new requirement creates the problem that the pattern is designed to solve. The resulting architecture is not claimed to be universally superior: it trades some additional classes and indirection for clearer separation of variable business rules.

Overall, the case study shows how a real Rails application can be analyzed, challenged, evolved and reproduced across languages while maintaining a clear distinction between framework mechanisms, domain design and recognized GoF patterns.

11 References

1. Andrew D. Silva, *rails-room-booking*, GitHub repository, baseline commit `5e15551ee646871b4bed32c982fa17da319d3ee3`. https://github.com/andrewdsilva/rails-room-booking

2. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

3. Ruby on Rails, *Rails Guides: Getting Started, Active Record and Active Support*, official Rails documentation.

4. Pundit, *Pundit Authorization for Ruby Applications*, project documentation.

5. Devise, *Flexible authentication solution for Rails*, project documentation.

6. Payify, payment integration dependency referenced by the baseline application.
7. Stripe, *Stripe API Documentation*, payment platform documentation.
8. PlantUML, *PlantUML Language Reference Guide*, documentation for editable UML diagrams.
9. Java Platform Documentation, OpenJDK API and language documentation.
10. Python Documentation, Python language and standard-library documentation.
11. Node.js Documentation, JavaScript runtime documentation.
12. ISO/IEC 14882, *Programming Languages – C++*, C++ language standard family.

A Detailed Source Evidence

A.1 CrudConcern evidence

The verified baseline `CrudConcern` contains the common CRUD algorithm and the hook methods used by the concrete controllers. The source is `code/app/controllers/concerns/crud_concern.rb:1-149`. The critical structural evidence is:

Listing 5: Core Template Method structure in CrudConcern

```
def create
  authorize object_class
  @object = object_class.new(form_values)

  if @object.valid?
    @object.save!
    redirect_to_show(@object)
  else
    render :new, status: :unprocessable_entity
  end
end

private

def base_class
  raise NotImplementedError
end

def object_class
  raise NotImplementedError
end
```

The same concern supplies `index`, `show`, `new`, `edit`, `update` and `destroy`. The concrete controllers then specialize the hook methods.

A.2 Booking evidence

The baseline Booking source establishes:

- `belongs_to :user`
- `belongs_to :room`
- validation of dates, room and user
- date-conflict validation
- payment state queries
- cancellation through `canceled_at`
- price access through `total_ttc`
- `after_save :create_payment`
- `after_create :notify_booking`

The cancellation implementation is intentionally simple:

Listing 6: Baseline cancellation behavior

```
def cancel!
  update_attribute :canceled_at, Time.now
end
```

This is why cancellation policy is treated as a new evolution requirement rather than as an existing baseline Strategy.

B Part 2 Change Acceptance

B.1 Change 1 acceptance

The implementation is considered structurally satisfied when a booking can be associated with breakfast or airport-transfer service objects/records and the service subtotal participates in the total calculation without modifying the original room amenity list.

B.2 Change 2 acceptance

The room-upgrade operation must reject an unavailable target room, calculate the target/base price difference, replace the booking room when valid, and recalculate the booking total.

B.3 Change 3 acceptance

The cancellation context must accept a policy, delegate the eligibility/refund operation to that policy, and support at least flexible, partial-refund and no-refund concrete policies without embedding all algorithms in the context.

C Cross-language Test Specification

The representative Strategy input is a booking amount of 430. Expected output is 430 for flexible cancellation, 215 for a 50% partial refund, and 0 for no refund. The Template Method example must demonstrate that a common workflow is reused while Booking, Room and Settings supply different resource-specific output.

The execution evidence stored under `Part3/evidence/` records the actual output for each language. The official `data/cross_language.csv` records source LOC, maximum function cyclomatic complexity, runtime/compiler, command, result, median process time and language notes.

D Submission and Evidence Audit

Table 15: Final evidence audit

Item	Evidence	Status
Team metadata	Master project brief	Verified
Baseline commit	Original GitHub repository	Verified
Baseline pattern count	`data/pattern_counts.csv` + source evidence	Verified
Template Method evidence	Source paths and participant mapping	Verified
Part 2 changes	Full evolved Rails application, migrations and RSpec receipts	36 examples, zero failures
Strategy implementation	Context, abstract contract and three concrete policies	9 service + 6 request examples passed
Part 3 tests	Eight snippets	Passed 1/1 each
Performance measurements	`data/cross_language.csv` + raw evidence	Verified
LLM records	`data/llm.csv` and retained prompts/results	Partial; historical metadata gaps disclosed
Prompt 02/03 verbatim history	Not retained	Not verified